PingRoom Final Implementation Review

MERN real time chat application

Prepared by Tijil Sharma, Riya Chopra and Vidhanshu Bhardwaj
Database Systems project
Final demonstration 20 October 2026

Abstract

PingRoom lets users send direct and group messages and return to their conversation history later. We use MongoDB for storage, Express and Node.js for the server, and React for the interface. Users, conversations and messages have separate collections. Each conversation stores its participants and a preview of the latest message. When a message changes, a transaction keeps that preview consistent with the message record.

The application supports account management, text messages, attachment links, search and paginated history. Socket.IO handles live events, including typing and presence. The local test run passed 13 API and database tests and 4 browser tests. The React build and source formatting checks passed too. We have not measured the latency target proposed in Review 1.

Final submission

Deliverable	Location
Application and source	github.com/TijilSharma/PingRoom
Final report	output/PingRoom_Final_Review.pdf and .docx
Pitch presentation	output/PingRoom_Final_Pitch.pptx
Code explanation and demo	docs/CODE_WALKTHROUGH.md and docs/DEMO.md
Reproducible verification	tests/api.test.js, tests/browser/chat.spec.js and GitHub Actions

Review basis

In Review 1, we set out the requirements and collection design. Review 2 covered the database implementation and the seed dataset of 138 documents. This submission describes the current code, the checks that passed, and the limits of the local setup.

1 Requirements and final scope

Requirement	Final implementation and evidence
FR 1 Accounts	Register, login, profile update and password-confirmed deletion. Bcrypt hashing and JWT authentication.
FR 2 Direct conversations	User search and direct-chat creation. Sorted participant pair plus a partial unique index prevents duplicates, including concurrent requests.
FR 3 Group conversations	Named groups with 3 to 50 members. Creator alone can rename or delete the group.
FR 4 Real time messaging	HTTP validates and persists messages. Socket.IO publishes committed message events to authenticated participant rooms.
FR 5 Message status	Sender begins in receipt arrays. Delivered/read require all participants. Repeated receipts are idempotent and cannot regress read status.
FR 6 Presence and typing	Per-user socket counts support multiple tabs. Typing events are transient, throttled and membership-checked.
FR 7 Message history	Newest-first query with a timestamp and ObjectId cursor. The client shows each page chronologically.
FR 8 Attachments	Validated HTTP(S) image/file links with display name. Binary upload and object storage are outside this implementation.
FR 9 Recent conversations	Conversation list sorted by updatedAt with cached lastMessage and aggregated unread counts.

Nonfunctional results

The browser tests check the interface at 390 × 844 pixels, alongside the desktop flows. API tests check authorization and transaction behavior. The sub-300 ms latency target from Review 1 still needs a load test. Our local replica set has one MongoDB member, so it supports transactions but cannot tolerate the loss of that member. We have not implemented sharding or horizontal scaling.

Final changes after Review 2

We corrected the development proxy to use the configured API port. We also fixed a receipt bug: opening Database Studio could mark messages in the selected conversation as read even though the chat was hidden. A browser test now checks this case. Dialogs support Tab navigation and Escape, and the supporting React components have separate files. The repository includes a GitHub Actions workflow for the verification commands.

2 Collections relationships and indexes

Collection	Document shape and rationale
users	username and email have unique indexes. passwordHash is excluded by default. avatarUrl, status and lastSeen hold profile/presence information. Timestamps record creation and updates.
conversations	type, title, ownerId, participants[], directKey and lastMessage. Participant ObjectIds form a bounded embedded membership array. Shared users remain referenced documents.
messages	conversationId, senderId, text, optional attachment, deliveredTo[], readBy[], status and editedAt. Each message has timestamps and belongs to one conversation.

Conceptual relationships

A user can join several conversations, and each conversation can contain several users. We represent that many-to-many relationship with participant IDs inside the conversation. Each message refers to one conversation and one sender. Mongoose can populate these references, but they are not database-enforced foreign keys. The API checks membership and handles related records when an account or conversation is deleted.

Index definitions

Collection	Index	Query or constraint
users	username: 1; email: 1	Two unique indexes protect account identity.
conversations	participants: 1, updatedAt: -1	Multikey compound index for membership-filtered recent conversations.
conversations	directKey: 1	Unique partial index where type is direct.
messages	conversationId: 1, createdAt: -1, _id: -1	Ordered history and stable cursor tie-breaking.
messages	text: text	MongoDB text search scoped by conversation.

Embedding and denormalization

We keep messages outside the conversation document because history has no fixed upper bound. Embedding every message would eventually run into the 16 MB BSON document limit. The small lastMessage snapshot is a deliberate duplication: it lets the sidebar show a preview without fetching a message for every conversation. Transactions keep the two copies consistent. Text search uses its own index, separate from the ordered-history index.

3 Application architecture and CRUD

Runtime structure

The production application runs in one Node process. It serves the built React files, the Express API and Socket.IO on port 3100. MongoDB runs in Docker on loopback port 27018 as replica set rs0. For development, Vite forwards API and socket requests to the server port set in the environment file.

Layer	Code entry point	Responsibility
React	client/src/main.jsx	Chat state, fetch requests, socket listeners and UI updates.
Components	client/src/components/	Authentication form, attachment form and Database Studio.
HTTP API	server/app.js	Validation, authentication, membership, CRUD and aggregation.
Realtime	server/realtime.js	JWT handshake, user rooms, typing and presence.
Persistence	server/models.js	Schemas, references, indexes and database connection.
Startup	server/index.js	Connect, reset stale presence, serve assets and handle shutdown.

CRUD coverage

Entity	Create and read	Update and delete
User	Register; login; current account; user search	Update username; password-confirmed account deletion.
Conversation	Create direct/group; list by membership	Owner renames groups; creator deletes conversation and messages.
Message	Send text/link; history and text search	Author edits/deletes; members acknowledge delivery/read.

Authorization boundaries

A valid JWT identifies the account making a request. The API then checks whether that account belongs to the conversation. It checks the creator before allowing conversation management, and the sender before allowing a message edit or deletion. An inaccessible conversation returns 404. Database Studio shows activity only for conversations the user belongs to. Global collection counts and index definitions remain visible to logged-in users for the database review.

4 Transactions queries and consistency

Sending and changing messages

Sending a message changes two documents: the message itself and the conversation preview. The route validates the content and starts mongoose.connection.transaction. It checks membership, inserts the message and calls snapshot using the same session. Only after commit does it emit the Socket.IO event. Edits and deletions also recalculate the preview inside a transaction. If one of these database operations fails, neither change commits.

Deletion behavior

Conversation deletion removes the conversation and its messages in one transaction. Account deletion is broader. It removes direct chats and groups owned by that account. In other groups, it removes the account's membership, messages and receipt references, then recalculates receipt status and the preview. The user must enter the password again before deleting the account.

Stable history pagination

History pagination uses both the timestamp and message ID. A timestamp alone would skip records when several messages share the same time. The next page therefore selects earlier timestamps, or the same timestamp with a lower ObjectId. The query sorts by createdAt and _id in descending order and fetches one extra row to check whether another page exists. The default page size is 30, with a maximum of 100.

Aggregation examples

Result	Pipeline behavior
Unread counts	$match limits messages to member conversations, excludes the sender and already-read messages; $group counts by conversationId.
Conversation activity	$group counts messages and characters; $lookup resolves the title; $project selects fields; $sort orders volume.
Daily volume and statuses	$facet computes multiple summaries from the same filtered input. $dateToString groups dates by day.
Query-plan inspection	explain with executionStats reports the winning plan, keys/documents examined and returned records. The integration test verifies IXSCAN.

The rollback test throws an error after writing to both collections, then checks that neither write remains. The pagination test inserts 65 messages with the same timestamp and checks every page for duplicate or missing IDs.

5 Real time behavior and security

Event lifecycle

Socket.IO verifies the token during the connection handshake and checks that the account still exists. The server places the connection in a room for that user. After a message commits, the API sends an event to the participants' rooms. Typing events also require a membership check. A timer disconnects each socket when its token expires.

Presence and receipts

The server counts open socket connections for each user. Closing one browser tab does not mark the user offline if another connection remains. The last disconnect updates status and lastSeen. Receipts use set union to avoid duplicate user IDs, then compare the participant list with readBy and deliveredTo. Read implies delivered. The client acknowledges reading only when the chat is visible, not while the tab is hidden or Database Studio is open.

Controls and boundaries

Control	Implementation and qualification
Password storage	Bcrypt with cost 12. Hashes are not returned in public account objects.
Request validation	Zod types, maximum lengths, ObjectId format and HTTP(S) attachment protocol checks.
Access control	Membership, ownership and sender checks on protected operations.
Abuse reduction	Helmet headers, 32 KB JSON limit, API/auth rate limits and typing throttling.
Session handling	One-day signed JWT in sessionStorage. Logout removes the local token, but there is no server-side revocation list.
Frontend rendering	React escapes text. Attachment links open with noopener and noreferrer. Links are external resources, not server-side uploads.

Production limitations

The local setup has one application process and one MongoDB member. Socket events are not replayed after a disconnect, so the client reloads stored history when it reconnects. End-to-end encryption, password recovery, calls, file storage and multi-region deployment are outside this version. Before exposing the app on the internet, we would need HTTPS, restricted database diagnostics, backups and monitoring, and would need to remove the demo accounts.

6 Implemented user interface

The React screen has a conversation list on the left and message history on the right. Users can search the open conversation, edit their own messages and send text or attachment links from the composer. Direct and group chats use the same message model. Figure 1 shows the running application.

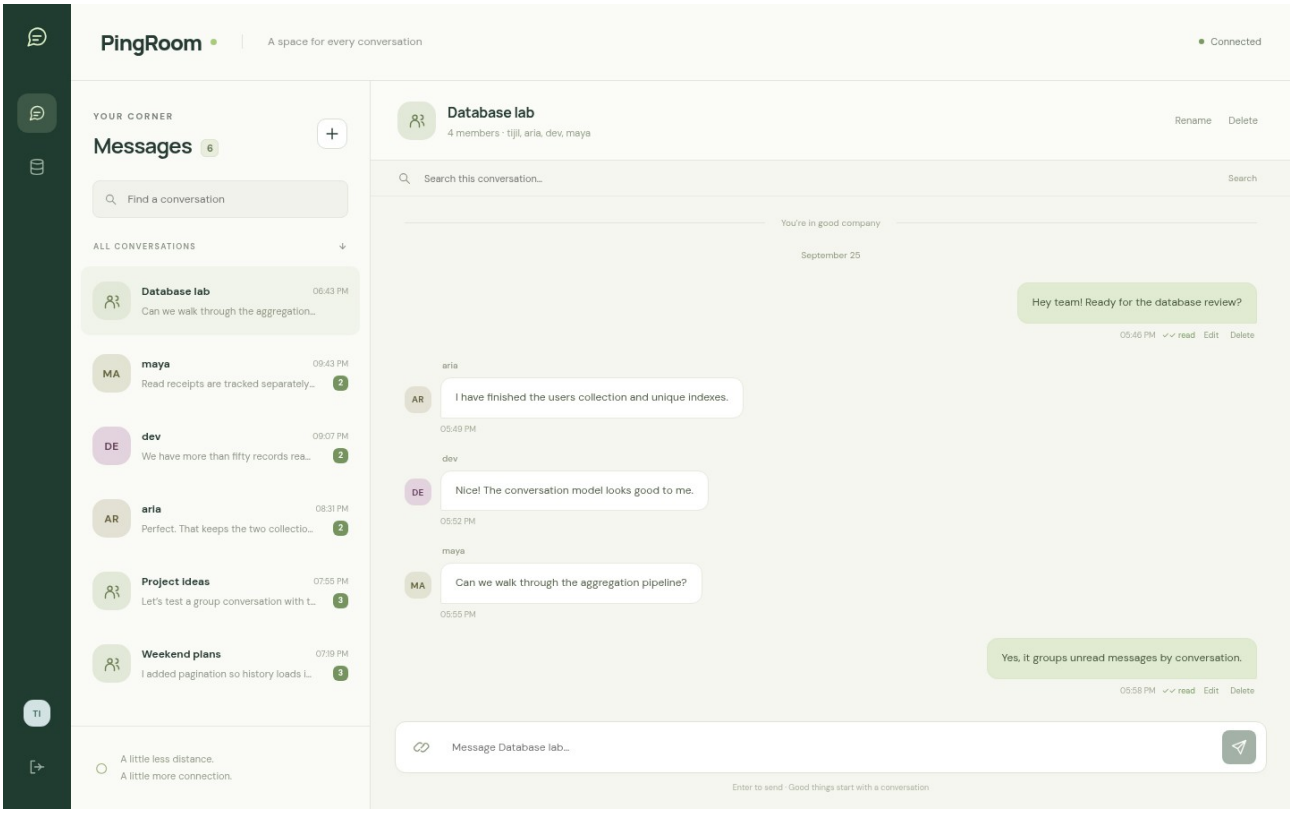


Figure 1 Conversation interface with membership, history, receipts and message actions.

Browser verification

The live-message test signs in as two demo users in separate browser contexts. One user sends a message, edits it and searches for it. The test checks that the second user sees the changes, then verifies that deleting the message removes it from the other screen. The remaining browser tests cover mobile layout, keyboard access to dialogs and the hidden-chat receipt bug.

Usability decisions

The interface shows connection state and request errors. It disables repeated send clicks while a request is pending and ignores old history responses after the user switches conversations. Typing indicators expire automatically. Authentication, attachment entry and Database Studio have separate React components so their code is easier to inspect.

7 Database evidence and sample data

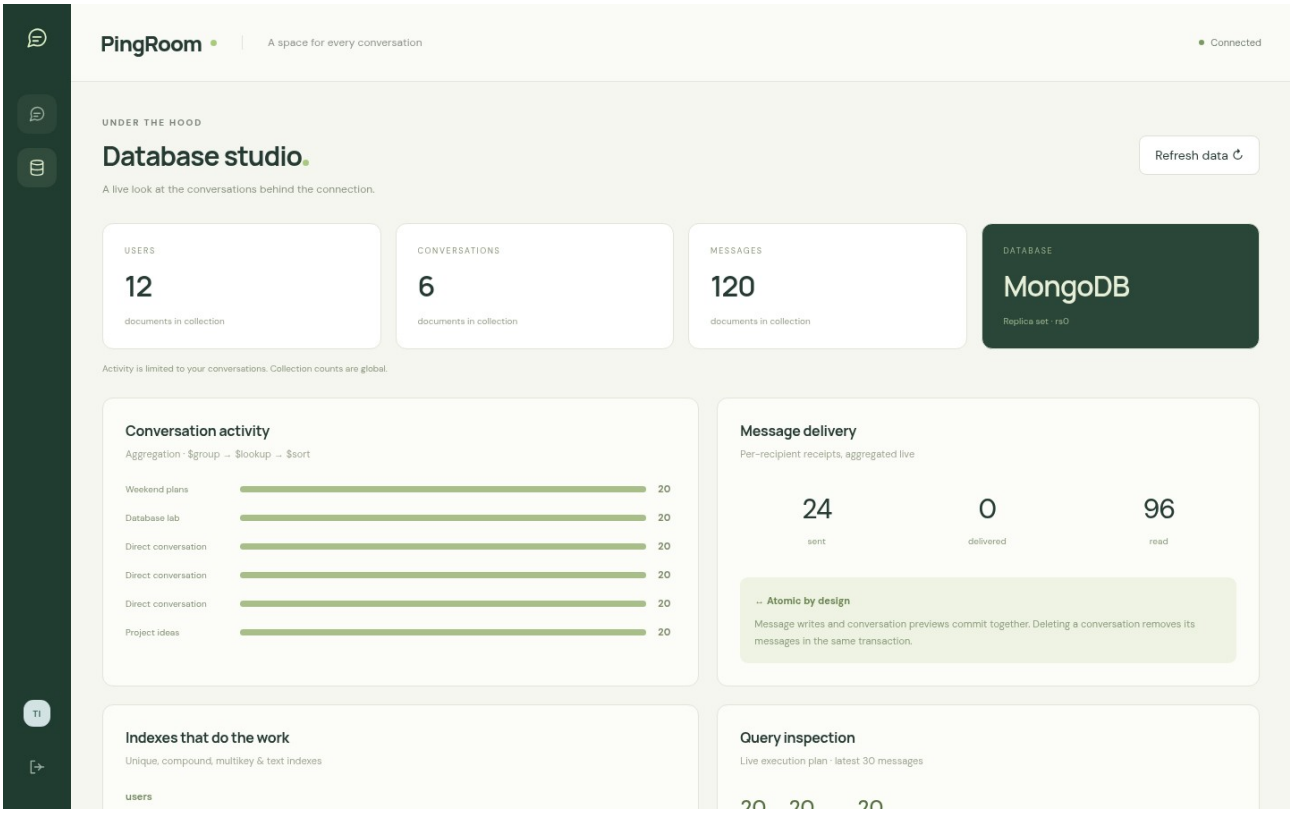


Figure 2 Database Studio showing live collection counts, activity summaries, indexes and query inspection.

Seed dataset

On an empty database, the seed script creates 12 synthetic users, 6 conversations and 120 messages in one transaction. That gives 138 documents. There are three groups and three direct chats, each with 20 messages. Running the script again leaves existing data alone. The seed test checks these exact counts in an isolated database, since the counts in the demo app can change during use.

Reading the database results

Database Studio reads the query plan and execution statistics from the running database. The activity summaries include only the user's conversations, while the collection counts are global. An IXSCAN shows that the selected query used an index. It does not establish the performance of every query. Likewise, rs0 confirms the replica-set configuration, but one member provides no failover.

Each seeded conversation has 20 messages, fewer than the default page size of 30. A UI pagination demonstration needs at least 11 more messages in one conversation. The automated test already checks multiple pages, including messages with tied timestamps.

8 Testing and running the project

Check	Observed final result
npm run build	Production React bundle built successfully.
npm run format:check	All checked source files passed.
npm test	13 passing tests, 0 failures.
npm run test:ui	4 passing browser tests, 0 failures using system Chrome.
npm audit	0 known vulnerabilities after the compatible dependency update.

Test coverage

The API tests check account and message operations, membership and ownership restrictions, receipt updates and account cleanup. They also check the database cases that are easy to miss in a manual demo: concurrent direct-chat creation, tied timestamps, rollback after a failed transaction and index selection. Socket tests cover live delivery, typing and presence across tabs. These checks do not measure load capacity or test failover, and they are not a penetration test.

Reproduction

To run the project, install Node.js 22.12 or later and Docker Compose, then run npm ci, npm run setup and npm start. Open http://localhost:3100. Setup generates a local JWT secret, starts MongoDB, seeds an empty database and builds React. For a two-user demo, sign in as tijil@vitstudent.ac.in and aria@vitstudent.ac.in in separate browser sessions. Both synthetic accounts use PingRoom@123.

Work required for deployment

A production deployment would need a multi-member replica set and shared presence state before adding more application servers. Load tests would be needed to assess the original latency target. File uploads would also require storage and access controls beyond the HTTP(S) links supported here.

References

Implementation references: server/models.js, server/app.js, server/realtime.js, client/src/main.jsx, scripts/seed.js and tests/. Earlier submissions: PingRoom Review 1 MERN ChatApp and PingRoom Review 2 Database Implementation. The verification results and screenshots in this report come from the local application on 19 October 2026. Repository: https://github.com/TijilSharma/PingRoom.